

SWMAL_Grp10-4

November 27, 2025

1 Aarhus Universitet — SWMAL E25

1.1 Assignment O4

1.1.1 Gruppe 10

1.1.2 Members

Name	Student ID
Per Hindbo Clausen	202007348
Mads Kampmann Elkjær	202209870
Daniel Them	201809623
Frederick Lykkegaard Nielsen	202207768

1.1.3 Hand-in Date

04/12-2025

2 Introduction

This report presents our final end-to-end machine learning project for O4. Our goal is to build an image-classification system that predicts the breed of a cat from a photo. We will focus on five breeds, Bengal, Domestic Shorthair, Maine Coon, Ragdoll and Siamese. We want to design and implement a pipeline, so that new breeds can be added with minimal changes.

We use the Kaggle Cats Breed dataset, which gives us real world images of five different breeds with variation in resolution, lighting and background.

In the report, we follow an end-to-end structure, which defines the problem, describe the dataset, our choice of machine learning methods, explaining how we process, train, and evaluate our model. We will also discuss performance metrics, risks of under- and overfitting, and the optimizations used to improve generalization. The report concludes with key results and reflections on possible future improvements.

3 Problem statement

The goal is to develop a machine learning model that can identify the breed of a cat from an image. Correctly recognizing cat breeds is a task that involves understanding visual features such as fur pattern, colors, shape and other characteristics. However, these features could possibly be subtle and difficult to distinguish, especially on breeds that look alike.

The main challenge of real-world images, is that they can vary in lighting, resolution, pose and background, which makes it harder for a model to focus on the important features of the cat, instead of background and lighting features. Our approach includes preparing and cleaning the dataset, selecting appropriate machine learning methods, training the model and evaluation its performance using relevant metrics. We want to create a model that can somewhat reliably predict the five target cat breeds and can be expanded to include more breeds in the future.

4 Dataset

4.0.1 a. Concept

We want to build a robust image-classifier that predicts a cat’s breed from a single photo. The initial scope covers five breeds—Bengal, Domestic Shorthair, Maine Coon, Ragdoll, and Siamese, but the architecture will be modular (Each cat breed has their own folder) so adding new classes later requires only dropping more folders into the dataset, re-running the split and re-training or fine-tuning. We normalize inputs consistently, apply conservative augmentations to improve generalization (flip/rotate/brightness/contrast/zoom), and track performance with accuracy and macro-averaged F1. To understand model behavior, we’ll use a confusion matrix and per-class metrics; this is especially useful for visually similar breeds or when classes are imbalanced. The end product is a saved model artifact plus a small script that takes an image path and returns top-1 prediction with confidence, and maybe top-k probabilities for transparency.

4.0.2 b. Dataset

We use the [Kaggle Cat’s Breed Dataset](#), which provides exactly the five classes we target and already follows a directory-per-breed structure. This structure is compatible with standard loaders fx. (Keras `image_dataset_from_directory`) and plays nicely with reproducible train/validation/test splits. The pipeline stays open to growth: if we later collect more images for these breeds or introduce new breeds, we simply add folders with the appropriate names and rerun the preprocessing/splitting steps. This makes it straightforward to scale both the data and the classification without rewriting code.

4.0.3 c. Dataset description

Each sample is a color photograph (commonly JPEG/PNG in RGB) with non-uniform resolution and aspect ratio. Labels are implicitly defined by folder names, which reduces the chance of meta-data drift but can still contain occasional label noise (mis-sorted images). In a quick exploration pass, we verify the directory layout, count the images per class, and check for corrupt files or near-duplicates using a perceptual hash. We look at global summaries like width/height distributions and aspect ratios to inform the target input size and resizing policy (e.g., pad/crop vs. stretch). We also visualize single-image RGB histograms and small grids of random examples per class to spot systematic differences such as background bias (studio shots vs. household environments), lighting

extremes, or watermark artifacts. To hold the scope of the project down we won't go through the pictures for background bias, as it might take a lot of time. There might be some issues which could be mild class imbalance, resolution variance, and varied backgrounds that can mislead the model into learning context instead of the cat.

4.0.4 d. Use of dataset

We'll use the dataset for multiclass classification: given a cat photo, predict one breed among our five classes. Classification fits because the target is a discrete label, not a numeric value (so not regression). We'll resize and convert to RGB to standardize inputs and stabilize training, then make stratified train/validation/test splits to get fair evaluation. For metrics, we pick accuracy for overall performance and macro-F1 so small classes count equally (We might add a cat breed with not a lot of pictures). We will use a confusion matrix that shows us which breeds the model confuses.

4.0.5 Sources:

- [Kaggle Cat's Breed Dataset](#)
- [scikit-learn: train_test_split](#) (stratified splitting)
- [scikit-learn: StratifiedKFold](#)
- [scikit-learn: StratifiedShuffleSplit](#)
- [scikit-learn: StratifiedGroupKFold](#)
- [scikit-learn: f1_score](#) (use `average="macro"`)
- [scikit-learn: confusion_matrix](#)
- [scikit-learn: ConfusionMatrixDisplay](#)
- [scikit-learn: classification_report](#)
- [scikit-learn: cross_validate](#) (multi-metric CV)
- [scikit-learn User Guide: Metrics & scoring](#) (classification metrics)
- [scikit-learn User Guide: Cross-validation](#)

We start by setting the dataset path, importing the libraries, and configuring plotting so the rest of the notebook knows where the images are and can visualize results consistently.

```
[1]: # Setup: Datapath and imports
DATA_DIR = "CatData/cat_v1"

from pathlib import Path
import numpy as np
import pandas as pd
from PIL import Image, UnidentifiedImageError
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (8,5)
plt.rcParams["axes.grid"] = True
```

Next, we scan the class folders, open each image, and record format, color mode, width, height, and aspect ratio to build a simple metadata table.

```
[2]: # Scan dataset (formats, color spaces, sizes)
IMG_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".gif", ".tif", ".tiff", ".webp"}

rows, bad = [], []
root = Path(DATA_DIR)
classes = sorted([d.name for d in root.iterdir() if d.is_dir()])

for cls in classes:
    for p in (root/cls).rglob("*"):
        if p.is_file() and p.suffix.lower() in IMG_EXTS:
            try:
                with Image.open(p) as im:
                    im.load()
                    w, h = im.size
                    rows.append({
                        "path": str(p),
                        "label": cls,
                        "format": im.format,      # JPEG, PNG, ...
                        "mode": im.mode,         # RGB, L, RGBA, ...
                        "width": w,
                        "height": h,
                        "aspect": (w/h if h else np.nan)
                    })
            except (UnidentifiedImageError, OSError):
                bad.append(str(p))

meta = pd.DataFrame(rows)
print(f"Classes: {classes}")
print(f"Total images: {len(meta)} | Unreadable: {len(bad)}")
meta.head()
```

```
Classes: ['bengal', 'domestic_shorthair', 'maine_coon', 'ragdoll', 'siamese']
Total images: 3649 | Unreadable: 0
```

```
[2]:
```

	path	label	format	mode	\
0	CatData/cat_v1/bengal/81cfd3adf43735cfb1ff26f...	bengal	JPEG	RGB	
1	CatData/cat_v1/bengal/Bengal_35.jpg	bengal	JPEG	RGB	
2	CatData/cat_v1/bengal/00e79f939696ea0c09560315...	bengal	JPEG	RGB	
3	CatData/cat_v1/bengal/boy-bengal-cat-names.jpg	bengal	JPEG	RGB	
4	CatData/cat_v1/bengal/Bengal_197.jpg	bengal	JPEG	RGB	

	width	height	aspect
0	1200	800	1.500000
1	300	306	0.980392
2	2554	3222	0.792675
3	1250	1351	0.925241
4	300	225	1.333333

Then we print a directory tree to document how the data are organized on disk, showing the root and the class subfolders.

```
[3]: # Representation on disk (folder structure)
def print_tree(path: Path, max_depth=1):
    def _walk(p, prefix="", depth=0):
        if depth > max_depth: return
        entries = sorted(p.iterdir(), key=lambda x: (not x.is_dir(), x.name.
↳lower()))
        for i, e in enumerate(entries[:50]): # truncate for readability
            is_last = (i == len(entries)-1)
            branch = "  " if is_last else "    "
            print(prefix + branch + e.name + ("/" if e.is_dir() else ""))
            if e.is_dir():
                _walk(e, prefix + (branch if is_last else "    "), depth+1)
    print(p.name + "/")
    _walk(path)

print_tree(root, max_depth=1)
```

```
1XAG3BWR6R6Y.jpg/
  bengal/
    00438-Bengal-cat-snarling.jpg
    00441-Very-timid-Brown-Spotted-Bengal-cat-ears-back.jpg
    00e79f939696ea0c095603154c4af840.jpg
    03915-Pair-of-Bengal-kittens-white-background.jpg
    04413-Bengal-cat-white-background.jpg
    047ffa2f7165ccc4585e418ddb800299.jpg
    05050-Brown-spotted-Bengal-cat-sitting-on-grey-background.jpg
    09a0c5d574f818471c84cf509fc786dc.jpg
    0c87060cea587a01f0a254066c95deaa.jpg
    106f59608bc9fc6ac11df31008efdf94.jpg
    1096165.jpg
    1111246.jpg
    1113154.jpg
    1142194.jpg
    119mokiL1.jpg
    1200px-Bangie_the_Bengal_Cat.jpg
    122433177_3649288731759742_6202950879910256561_o.jpg
    12651963_f520.jpg
    14105889_f520.jpg
    15658-Brown-spotted-Bengal-female-cat.jpg
    15a0a84bd11fcae601ed41f5672191ba.jpg
    1aeae43be56c47450ef4a3354d3e940d.jpg
    1b2a68f85a32410fb3fd776927a2f4ef.jpg
    1e8351aa322b32f7a23980d7cbed1a8.jpg
    20121130172827.jpg
    22384097_10154807489365927_7732210437813358666_o.jpg
```

2308642374_57489bb2ce_o-57b742c15f9b58cdfdb97b03.jpg
 2385251605b4059a955e526.09727891-profile-1422.jpg
 28157986_217164348841297_8045674480825008128_n-5a9606581f4e1300367fff3f.jpg
 28209b6d36d53298996458299227dca4.png
 28430549_572162953143506_2215371262585208832_n-5a96060b1f4e1300367ff4f0.jpg
 306059656_c4fafc4a87_o.jpg
 32249-Bengal-female-cat-white-background.jpg
 379800.jpg
 3931eb090308eeaa58719fe66b9926c3.jpg
 3be35b792cbaa987776772fd2a67b78e.jpg
 3eb7e9a7542bdead2c633e4a2b06a588.jpg
 40706-Portrait-of-Bengal-cat.jpg
 40725-Bengal-cat-sitting-white-background.jpg
 425c7c6eba0b92a7da8d32e4502ff0e5.jpg
 42cfe991ab15fe999d446660b4ccb0b8.jpg
 53653d0ade9fc4d625055cd5c45b8d5c.jpg
 5fba2bc61c502d3177e845b48807e38d.jpg
 60a954c22bba1.jpg
 610850dbd28f3cfea67e7d6350a8a0c1.jpg
 677597.jpg
 67dd59a71ac467ca3224dd3ce87b1a6e.jpg
 687362.jpg
 68a0ecf22205aca18f080c83a32e083e.jpg
 6ae66c9e98081d48648b8ecc9784aedef.jpg
 domestic_shorthair/
 046dce16530c0e2f62348adc724dbc4b.jpg
 04f2e5e2f67443c7c3299ce4078dc2a2.jpg
 0C40F888-7B22-4B2C-88B5-049BBB40E374.jpeg
 0cd53884802ed1f687f6ddb860973b8b.jpg
 10bc40c87f24b7442f454f90f1248e16.jpg
 1200-93358197-domestic-shorthair-cats-playing.jpg
 1200px-American_Shorthair.jpg
 1200px-Cat03.jpg
 1200x0 (1).jpg
 1200x0 (2).jpg
 1200x0 (3).jpg
 1200x0.jpg
 1280-594481594-little-kitten-scaled.jpg
 1286px-British_Shorthair_Cat.jpg
 13299451414_6217c5c653_b.jpg
 1585003597_2615038995e793c4deb2bb.jpeg
 17e3677e829dc216edcc4c1982680495.jpg
 189edc1cdd4d2ad45352e6d5a05ebdf2.jpg
 19072-British-Shorthair-tabby-tortoiseshell-cat-reclining-white-background.jpg
 19c64d2430d89fb1d0b21f3037b52431.jpg

20-02-2020RG-3-scaled.jpg
 20171217_153823.jpg
 20180808_221155.jpg
 204574584.jpg
 204646681.jpg
 210084761.jpg
 228794488.jpg
 232069151.jpg
 242111_e8cda_orig.jpg
 31f7662ca0182a2ef145e580745a1a01.jpg
 37087816_10216998859910320_332375493129011200_n.jpg
 388271506.jpg
 404469246.jpg
 438649_28986_orig.jpg
 461138744.jpg
 466748758.jpg
 482497577.jpg
 499844134.jpg
 501189109.jpg
 502158847.jpg
 504925860.jpg
 508396792.jpg
 509787583.jpg
 50f20a3651b0742f2eb623a5c011a653.jpg
 523771009.jpg
 536180_dd3d3_orig.jpg
 543426094.jpg
 546250_9f2e0_orig.jpg

 5bdf4ba6-184b-4559-88e3-8cab3b9d89a_5.a15ed7d25a297e1bf4776d6d5691a422.jpeg
 5bec89c5926a1.image.jpg
 maine_coon/
 04451e0620bad89784005f3443dd7fed.jpg
 06a71c9902b7dcbb9f741d0599baf907.jpg
 098d3da3d30a35356744d87c57af4280.jpg
 11-25.jpg
 1280-491043662-tabby-maine-coon-cat.jpg
 14052241_f120.jpg
 14134300_f1024.jpg
 154d0dfcc8a953dc2adea273e02c0cb2.jpg
 1692_191118161233Q7_104.jpg
 17826-Maine-Coon-cat-white-background.jpg
 2003-4288-2848-dsc-8088-2e700.dsc-8088.htm
 20180512_031837-e1530464710292.jpg
 21667-Maine-Coon-kitten-white-background.jpg
 21911f4f1af72470cdf93bcfe4ca4591.jpg
 250a50bcb61c258523571a78c63b787.jpg
 2790ae2c5f7e4ff5e12346fe0eb2b4ab.jpg

28385-Maine-Coon-kitten-white-background.jpg
29769a4f1c7ddf57d81b5d4bfbd084b4.jpg
2sv63h8w3pt21.jpg
30859230298_aea362e94d_b.jpg
3506c008f54e869aa77e405e089550b5.jpg
36f7f53a0f1b70a126167f31f7a114ca.jpg
39728-Tabby-Maine-Coon-cat-white-background.jpg
39730-Tabby-Maine-Coon-cat-white-background.jpg
3a8208778a3718a73723b93768cfd8da.jpg
3adb3fe3adb0ab2e044e6e3ba2469a54.jpg
422600045dc93960e3ef2feb104f56b0.jpg
42e8f81f485608e6899316ccac139a20.jpg
45aaec1146c25dac8c832190e6057401.jpg
4912015294ef862a23196b23de8a17d6.jpg
4c67d602534c656db0997a7bdc72c276.jpg
4d7499c04dcc3706dbbc4d95be2fed9e.jpg
4fc046f9b86645a27c1de1207f1dc2e5.jpg
502154.jpg
51016dfcc8b60bb708b34ba27b92a051.jpg
5a40909340d5251a5d24df5fca8ff16d.jpg
5d06a126825383a5725a7251f747cdfd.jpg
5d816695a728c4d4b49306719951c29f.jpg
5ea0417ae1b850f3450b0a1420c6a214.jpg
5EEC667B-3B20-46B2-A162-ABDEEC075F69.jpeg
69a5c257eede39e548ccbb4b23006387.jpg
6a791bc868d7942b58d2feea5c3c8894.jpg
6b33dcc2c854f54de0ae6f6f8ff72eaf.jpg
6e4833371387866c23c15259c60f10f0.jpg
7728082365d689027aa2c52.10203604-breed-344.jpg
813c64fa4a26a08852379a5261956886.jpg
84a0996fb6460e2249187fba9f125fee.jpg
84c440da9f7f78bdb8bc94b6ca6c332c.jpg
85011c8f18fd41087cab8f7fbf84419b.jpg
854d769e2b670e1f95d06ee690d411bb.jpg
ragdoll/
04ecca85dae9ae1f5f404e68a289139b.jpg
0599222d68f89df6a17784997ad4c816.jpg
0a8d33cd127ba447efd93b9488585790.jpg
0a9df4c05aff675b1426c701fea9e0dc.jpg
0c8e50eb24db1849c1a0ecc638b9ba5b.jpg
11213-Tortoiseshell-and-two-ragdoll-cats.jpg
11e43d6d946b44abc868901dbdfdc7b5.jpg
1200-37427470-ragdoll-cat.jpg
124199014_xl.jpg
1732624227ee9a0bb58e702d6a2d9f3f.jpg
181124608_a18f0d495e_b.jpg
19cf174d39b6e284546bd3c9dde2617d.jpg
1ef5b49ffff19251b56c7786486f84ba.jpg

1fa7036535595c67b52c0bb57764d838.jpg
20648-Lilac-bicolour-Ragdoll-cat-lying-with-head-up.jpg
21bcf0a08369291d071821a629b60258.jpg
22219924_719461648244213_4942048061401934211_o.jpg
2751140.png
27665-Blue-eyed-Ragdoll-kitten-white-background.jpg
276bdce103d8590fd3ebc2cf7acf1ba6.jpg
2cf008151de12819f4ebad7e5c24a266.jpg
2e2cc888-7cdd-6404-f139-79c1f201d23b.jpeg
2e3f652667c93ac2dd4c5eb2760c93fe.jpg
38773-Ragdoll-kitten-among-spring-flowers.jpg
39421-Blue-eyed-Ragdoll-kitten-white-background.jpg
39772-Ragdoll-cat-white-background.jpg
3a189f490f7dd30e3a83a8f677523c9e.jpg
3b8a614226a953a8cd9526fca6fe9ba5.jpg
3fe47f4244148fcdc9cfd8d8c3e44dda.jpg
41534-Ragdoll-kitten-10-weeks-old-white-background.jpg
43ebf72b13de09361d7a52878773d.jpg
49b826e113fb2acea171222930648c5c.jpg
49dd1d27a854c4b44b72932aa208d04e.jpg
4dfc69125aa9cfafb5c7333ac2211b6e.jpg
549162.jpg
56902602_414038916065155_4512174701024918698_n-
95d7a6c3150e46be804a19a821abeceb.jpg
5ab4f4dbd3b6a5fb3ff4af08a31f0dbf.jpg
5be919c106a1a2c4681972f940121a9a.jpg
5dbd9785be54163365380837dec7b77.jpg
60841aa3bf294bb6bcb203a0e8d9a191.jpg
63d5612af077b1d2f62687467860bdd4.jpg
68636f1eed8430aa99bcb300eea98bf.jpg
69d6575461a0abe33a995c7792335.jpg
6a02fbf286bc2e5cde7064dc0e28d8c5.jpg
6e5a57aadb71f4635e6f99956a14772.jpg
72b906a708de17740617e3d907b802d2.jpg
741582.jpg
7536708_XXL.jpg
76911a3bb79a442c891d113e0156e086.jpg
77466c59fdd381902793db0f5ac22ec5.jpg
siamese/
01FUL1T2FSY8.jpg
02BQ9G8GRJ5G.jpg
02NWWAHTQGxW.jpg
02V221B4MR5L.jpg
032VXIeAY2ND.jpg
04QFCY4S075A.jpg
059QWE3U7U3Q.jpg
059Y1KGDxHXA.jpg
06BN2IHH0N35.jpg

06C3ICEWJQ7Y.jpg
0706f1a90468dec8fe5f2d00f1a33f4e.jpg
07N4L8J7VX74.jpg
08J104YWFY6A.jpg
09ZV6J8EK49K.jpg
0_GLP_SAH_SM190278_30JPG.jpg
0a92a43b384b5f41931afa751601210f.jpg
0AG1KKQIHG0C.jpg
0AHTZA100DLH.jpg
0B76UT4VATNW.jpg
0CMMOVEG1RYP.jpg
0CVW8UTW8XPB.jpg
0FMQHNWS8YQR.jpg
0FV04P1M1Z89.jpg
0FZOAPF3624B.jpg
0G0PQREKPXFB.jpg
0GUVA5EEJ10H.jpg
0HQAP0YUTXDX.jpg
0INN99LL81SY.jpg
0IWHBN5CMMRU.jpg
0KD704S81NDD.jpg
0LXYEI4K6VYA.jpg
0M69ENIUUV16E.jpg
0MJ0BNRCPJF1.jpg
0NVHB58JVZIG.jpg
009RHU81QW6V.jpg
00P58JD03N5L.jpg
0P0VPJGPPPYV.jpg
0PVBVWCK47KG.jpg
0QZYSU91SKQR.jpg
0RVEY6ADRB1X.jpg
0S7B8Z7LV5WL.jpg
0T28H1DM8366.jpg
0T9NIKLALJ2S.jpg
0TGPDFM0MW4F.jpg
0UE1RTPH5658.jpg
0UZXYIZB60T3.jpg
0WDEQ7ANHIYY.jpg
0YLAMIS1EQOR.jpg
0YPCK3P9ANI7.jpg
0YQTSNNN3XGB.jpg

After that, we show class counts and simple tables of image formats and color modes. This checks how balanced the classes are and what kinds of files we have.

```
[4]: # Basic tables
      # Class counts
      cls_counts = meta["label"].value_counts().sort_index()
```

```
display(pd.DataFrame({"count": cls_counts}))

# Formats & color spaces
display(meta["format"].value_counts().to_frame("count").rename_axis("format"))
display(meta["mode"].value_counts().to_frame("count").rename_axis("mode"))
```

	count
label	
bengal	527
domestic_shorthair	520
maine_coon	540
ragdoll	557
siamese	1505

	count
format	
JPEG	3639
PNG	7
MPO	2
WEBP	1

	count
mode	
RGB	3644
RGBA	2
P	2
L	1

We then compute size stats per class and plot histograms of width, height, and aspect ratio. This tells us how big images are and helps if we want to choose a standard resize.

```
[5]: # Size statistics + histograms (width/height/aspect)
size_stats = (meta
    .assign(megapix=(meta["width"]*meta["height"])/1e6)
    .groupby("label")[["width", "height", "aspect", "megapix"]]
    .agg(["count", "min", "median", "mean", "max"]))
display(size_stats)

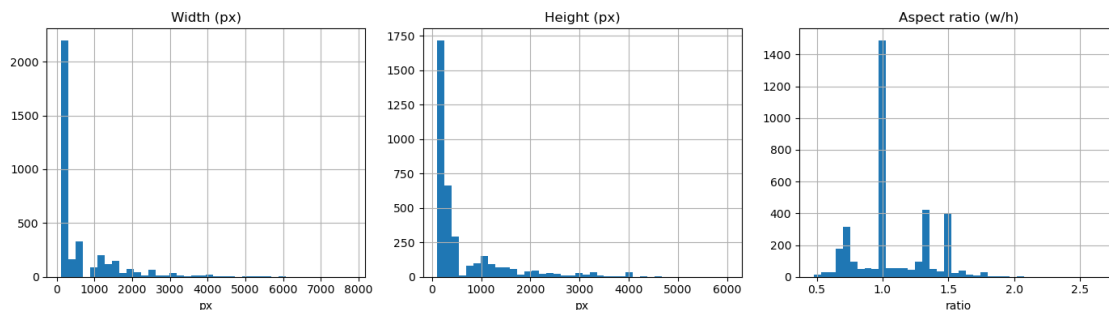
fig, axes = plt.subplots(1,3, figsize=(14,4))
axes[0].hist(meta["width"], bins=40); axes[0].set_title("Width (px)");
    ↪axes[0].set_xlabel("px")
axes[1].hist(meta["height"], bins=40); axes[1].set_title("Height (px)");
    ↪axes[1].set_xlabel("px")
axes[2].hist(meta["aspect"], bins=40); axes[2].set_title("Aspect ratio (w/h)");
    ↪axes[2].set_xlabel("ratio")
plt.tight_layout(); plt.show()
```

width				height				
count	min	median	mean	max	count	min	median	\

label									
bengal	527	192	500.0	888.573055	6016	527	145	375.0	
domestic_shorthair	520	190	300.0	881.950000	7793	520	148	400.0	
maine_coon	540	116	500.0	841.179630	5616	540	155	500.0	
ragdoll	557	172	500.0	932.976661	6000	557	112	456.0	
siamese	1505	256	256.0	500.795349	6000	1505	256	256.0	

		aspect						
	mean	max	count	min	median	mean		
label								
bengal	743.022770	4928	527	0.496000	1.333333	1.247432		
domestic_shorthair	865.788462	5472	520	0.483871	1.000000	1.042985		
maine_coon	832.737037	6000	540	0.483871	1.000000	1.065800		
ragdoll	861.725314	5389	557	0.476948	1.259214	1.160972		
siamese	469.294352	4630	1505	0.495495	1.000000	1.026400		

		megapix					
	max	count	min	median	mean	max	
label							
bengal	2.173913	527	0.031872	0.167000	1.368529	24.064000	
domestic_shorthair	2.027027	520	0.044400	0.120000	1.684241	40.484635	
maine_coon	2.183406	540	0.017980	0.185250	1.324973	25.000000	
ragdoll	2.678571	557	0.028552	0.187500	1.510091	24.000000	
siamese	2.370370	1505	0.065536	0.065536	0.671737	24.000000	



Next, we show a small grid of example images for each class with their labels. This is a quick visual check that the data look right.

```
[6]: # Examples & labels (image grid)
def show_examples(df, per_class=4):
    labels = sorted(df["label"].unique())
    nrows, ncols = len(labels), per_class
    fig, axes = plt.subplots(nrows, ncols, figsize=(2.8*ncols, 2.8*nrows))
    if nrows == 1: axes = np.array([axes])
    for r, lab in enumerate(labels):
        subset = df[df["label"]==lab]
```

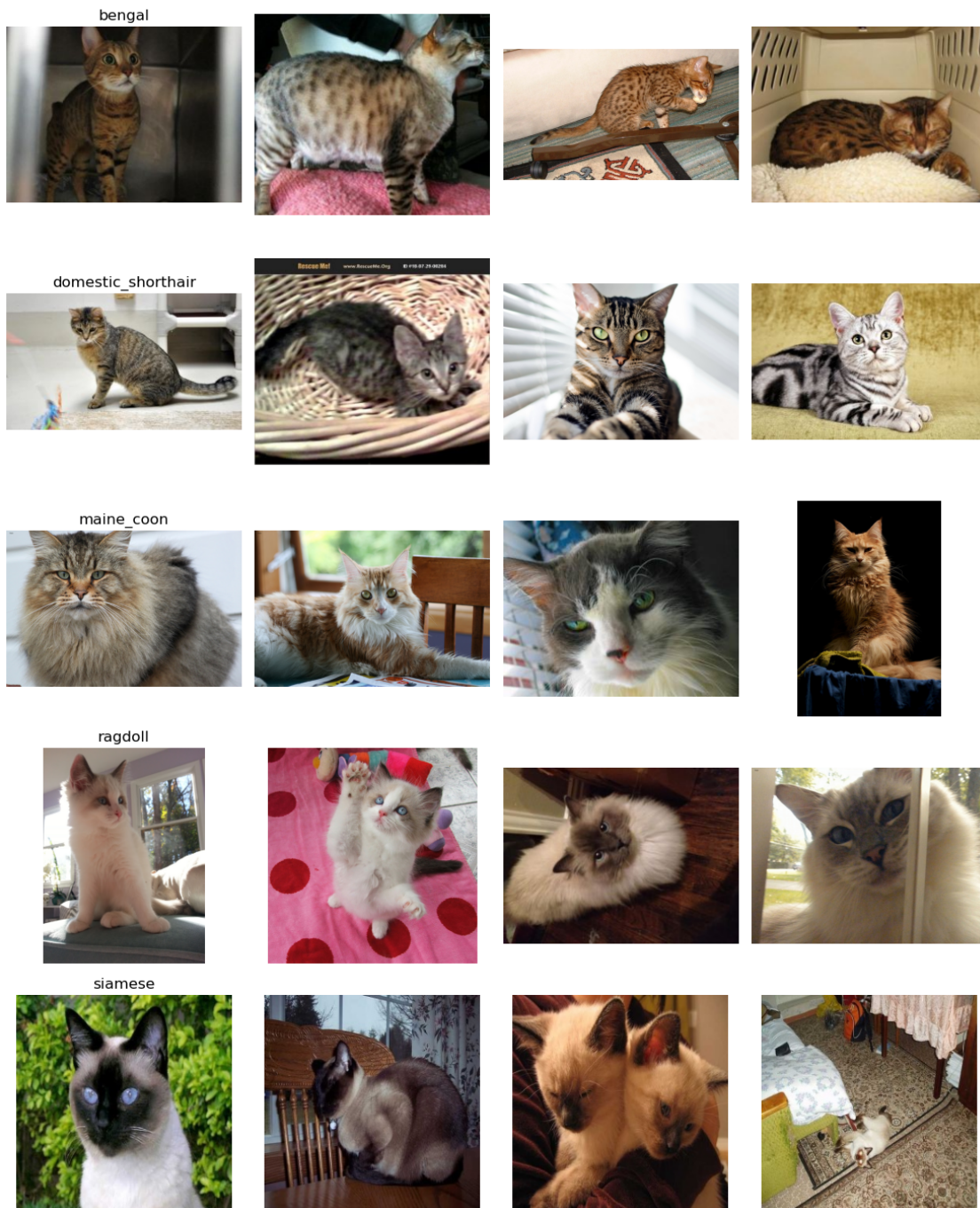
```

sample = subset.sample(min(per_class, len(subset)), random_state=42)
for c, (_, row) in enumerate(sample.iterrows()):
    ax = axes[r, c] if ncols>1 else axes[r, 0]
    img = Image.open(row["path"]).convert("RGB")
    ax.imshow(img); ax.set_axis_off()
    if c == 0: ax.set_title(lab)
plt.suptitle("Example images per class", y=0.995)
plt.tight_layout(); plt.show()

show_examples(meta, per_class=4)

```

Example images per class



We also plot RGB histograms for one image per class. That gives a basic view of brightness and color for single images.

```
[7]: # Histograms on individual images
def plot_image_and_histogram(image_path, bins=32):
```

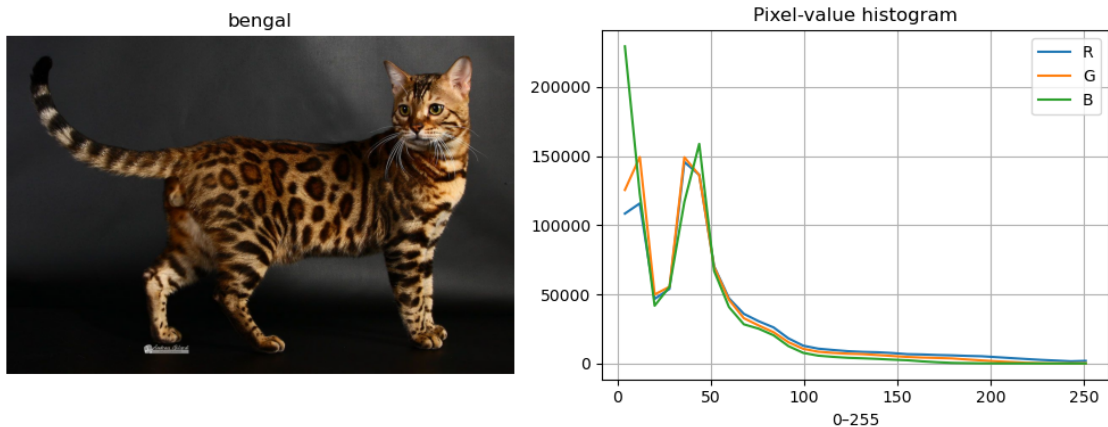
```

im = Image.open(image_path).convert("RGB")
arr = np.asarray(im, dtype=np.uint8)
fig = plt.figure(figsize=(10,4))
ax1 = plt.subplot(1,2,1)
ax1.imshow(im); ax1.set_axis_off(); ax1.set_title(Path(image_path).parent.
↪name)
ax2 = plt.subplot(1,2,2)
for ch, name in enumerate(["R", "G", "B"]):
    hist, edges = np.histogram(arr[...,ch].ravel(), bins=bins,
↪range=(0,255))
    centers = 0.5*(edges[1:]+edges[:-1])
    ax2.plot(centers, hist, label=name)
ax2.set_title("Pixel-value histogram"); ax2.set_xlabel("0-255"); ax2.
↪legend()
plt.tight_layout(); plt.show()

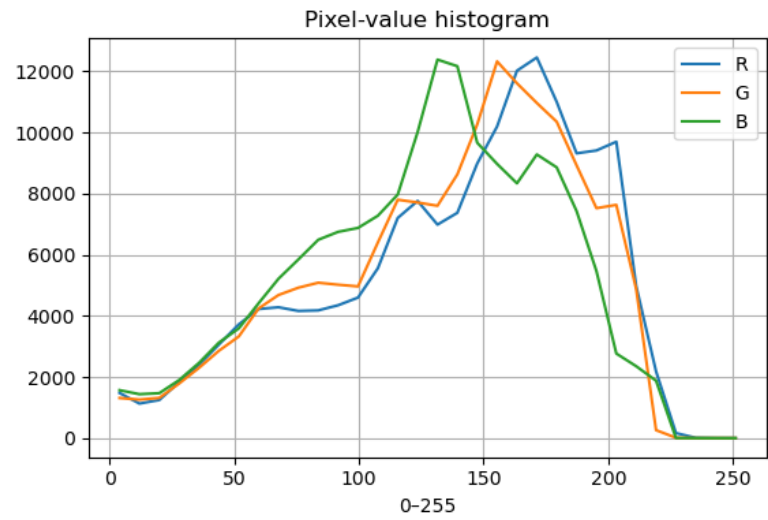
print("Single-image histograms (one per class):")
for lab in sorted(meta["label"].unique()):
    ex = meta[meta["label"]==lab]
    if len(ex):
        plot_image_and_histogram(ex.iloc[0]["path"], bins=32)

```

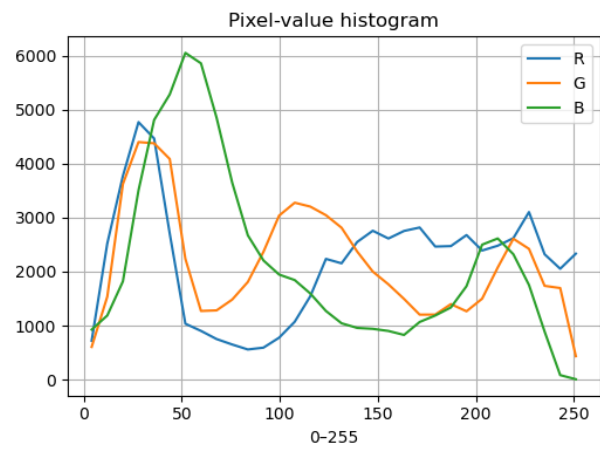
Single-image histograms (one per class):



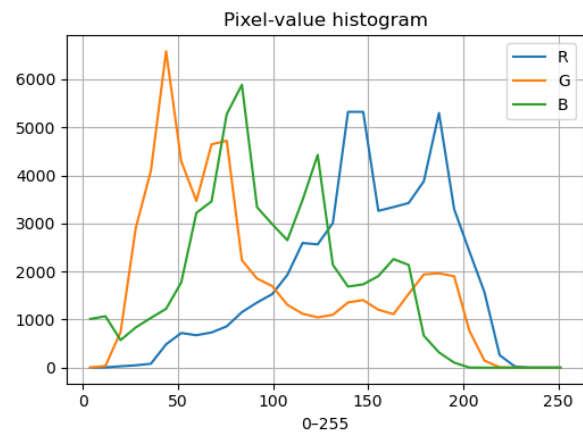
domestic_shorthair

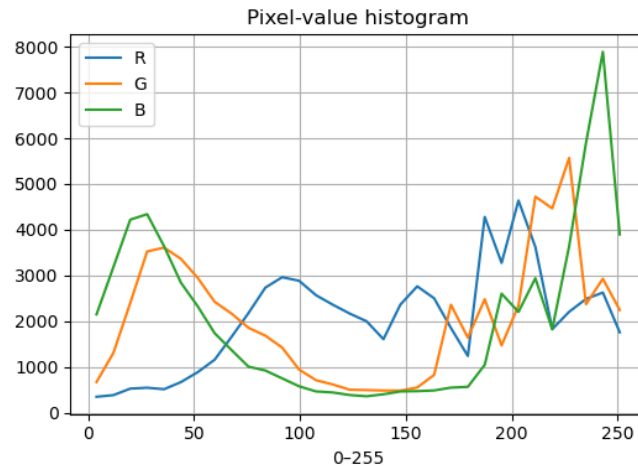


maine_coon



ragdoll





The above data analysis gives us a clear picture of what the dataset includes. We know the folder layout for loaders, the class balance, typical image sizes and aspect ratios, and the formats/color modes. Hopefully this will help us be more prepared for the later challenge of actually training a model.

```
[8]: import shutil
from PIL import Image, ImageOps

def standardize_images(
    df,
    out_root="CatData/_tmp_standardized",
    target_size=(299,299),
    out_format="JPEG",
    quality=95,
    preserve_structure=True,
    letterbox=False,
    bg_color=(0,0,0),
    val_fraction=0.10,
    test_fraction=0.10,
    random_state=42,
    balance_to_min_class=False, # <--- NEW
):
    """
    df: metadata dataframe with columns 'path', 'label'
    ...
    """

    assert 0 <= val_fraction < 1
    assert 0 <= test_fraction < 1
```

```

    assert val_fraction + test_fraction < 1, "val_fraction + test_fraction must_
↳ be < 1"

df = df.copy()

# --- NEW: balance to smallest class ---
if balance_to_min_class:
    counts = df['label'].value_counts()
    min_count = counts.min()
    df = (
        df
        .groupby('label', group_keys=False)
        .apply(lambda g: g.sample(min_count, random_state=random_state))
        .reset_index(drop=True)
    )
    print(f"Balanced dataset: {len(df)} images "
          f"({min_count} per class, {df['label'].nunique()} classes)")

# Clean output directory
out_root = Path(out_root)
if out_root.exists():
    shutil.rmtree(out_root)
out_root.mkdir(parents=True, exist_ok=True)

rng = np.random.default_rng(random_state)

# Decide train/val/test assignment
rand_vals = rng.random(len(df))

df["split"] = "train" # default
df.loc[rand_vals < test_fraction, "split"] = "test"
df.loc[
    (rand_vals >= test_fraction) & (rand_vals < test_fraction +_
↳ val_fraction),
    "split"
] = "val"

counts = 0
for _, row in df.iterrows():
    src = Path(row["path"])
    label = row["label"]
    split = row["split"]

    try:
        with Image.open(src) as im:
            im = im.convert("RGB")

```

```

        if letterbox:
            im.thumbnail(target_size, Image.Resampling.LANCZOS)
            canvas = Image.new("RGB", target_size, bg_color)
            off_x = (target_size[0] - im.width) // 2
            off_y = (target_size[1] - im.height) // 2
            canvas.paste(im, (off_x, off_y))
            out_im = canvas
        else:
            out_im = ImageOps.fit(im, target_size, Image.Resampling.
↳LANCZOS)

        split_dir = out_root / split
        subdir = split_dir / label if preserve_structure else split_dir
        subdir.mkdir(parents=True, exist_ok=True)

        out_name = src.stem + "." + ("jpg" if out_format.
↳upper()=="JPEG" else out_format.lower())
        out_path = subdir / out_name

        save_kwargs = {}
        if out_format.upper() == "JPEG":
            save_kwargs.update({"quality": quality, "optimize": True})

        out_im.save(out_path, out_format, **save_kwargs)
        counts += 1

    except Exception as e:
        print(f"Failed {src}: {e}")

    print(f"Standardized {counts} images -> {out_root}")
    print(f"Train size: {(df['split']=='train').sum()}")
    print(f"Val size:   {(df['split']=='val').sum()}")
    print(f"Test size:  {(df['split']=='test').sum()}")

    return df

meta_split = standardize_images(
    meta,
    target_size=(299, 299),
    out_format="JPEG",
    letterbox=False,
    val_fraction=0.10,
    test_fraction=0.10,
    balance_to_min_class=True,
)

```

/tmp/ipykernel_2670289/1203097361.py:36: DeprecationWarning:

DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
.apply(lambda g: g.sample(min_count, random_state=random_state))
```

Balanced dataset: 2600 images (520 per class, 5 classes)

Standardized 2600 images -> CatData/_tmp_standardized

Train size: 2060

Val size: 282

Test size: 258

5 Choice of ML algorithm

For this project we frame cat-breed recognition as a supervised multiclass image-classification problem as each input image is paired with a known breed label, and the model must assign one of five possible classes (Bengal, Domestic Shorthair, Maine Coon, Ragdoll, Siamese). Because the input data are images with a clear two-dimensional spatial structure, we use computer-vision methods rather than generic tabular algorithms. Convolutional neural networks (CNNs) are the standard choice for image recognition on large datasets such as ImageNet.

CNNs are well suited to this task because they exploit the topology of image data through local receptive fields and weight sharing. This allows the network to learn translation-invariant features, so if the cat is shifted, rotated, or appears at a different scale, the same convolutional filters can still detect relevant visual patterns such as edges, fur textures, and facial structures. Instead of relying on hand-crafted features like manually defined color histograms or shape descriptors, the CNN automatically learns a hierarchy of representations, from low-level edges and corners in the early layers to higher-level concepts such as ears, eyes, and overall head shape in deeper layers. This makes CNNs more robust to variations in pose, lighting, and background that characterize real-world cat photos.

Training a deep CNN from scratch usually requires very large labeled datasets and substantial computational resources. Our cat-breed dataset is much smaller (952) than ImageNet (14 million), so directly learning all weights from random initialization would increase the risk of overfitting and lengthen training time. To address this, we adopt transfer learning, which is standard practice for image classification when data are limited. We reuse a pre-trained CNN, Inception v3, which is originally trained on more than one million images and 1,000 object categories from the ImageNet dataset. Inception v3 is a deep, optimized CNN architecture that combines convolutional, pooling, and “Inception” modules to achieve strong accuracy without using too much computational powers.

In our transfer-learning setup, the lower and middle layers of the pre-trained Inception v3 model are frozen, because they capture generic visual features such as edges, textures, and simple shapes that are useful for almost any image domain. Only the final classification layers are replaced and retrained on our five cat-breed classes. This approach drastically reduces training time and data requirements while typically improving generalization compared with training a similar-sized network from scratch on a small dataset.

We implement our model in Python using TensorFlow and its high-level Keras API. Keras provides a simple way to define, train, and evaluate deep learning models, while TensorFlow handles the

underlying computation efficiently on both CPU and GPU. This framework also offers built-in support for loading pre-trained models such as Inception v3 and makes it straightforward to fine-tune them for our specific cat-breed classification task.

5.1 Sources

- [Szegedy, C. et al. “Rethinking the Inception Architecture for Computer Vision.” CVPR, 2016.](#)
- [ImageNet Project. “About ImageNet.”](#)
- [Kaggle. “Inception V3 Model”](#)
- [Tensorflow Learn. “Convolutional Neural Network \(CNN\)”](#)
- [Wikipedia. “Convolutional Neural Network.”](#)
- [Wikipedia. “Inception \(deep learning architecture\).”](#)
- [Keras: Transfer learning & fine-tuning”](#)

6 ML Data processing

her

7 Performance metrics

Here

8 Under- and overfitting

here

9 Optimizations and improvements

9.1 Confusion matrix

10 Conclusion

Here

11 KODE

```
[9]: import tensorflow as tf
from tensorflow.keras.applications import InceptionV3
from tensorflow.keras import layers, models
```

```

NUM_CAT_BREEDS = 5
width, height = 299, 299

base_model = InceptionV3(
    weights="imagenet",
    include_top=False,
    input_shape=(width,height,3)
)

```

```

2025-11-27 16:47:45.804316: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:47:45.805591: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:47:45.806921: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:47:45.808107: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:47:45.809267: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:47:45.810454: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:47:45.821200: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful

```

NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

2025-11-27 16:47:45.822402: I

external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful

NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at

<https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

2025-11-27 16:47:45.823590: I

external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful

NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at

<https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

2025-11-27 16:47:45.824791: I

external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful

NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at

<https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

2025-11-27 16:47:45.825965: I

external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful

NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at

<https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

2025-11-27 16:47:45.827110: I

tensorflow/core/common_runtime/gpu/gpu_device.cc:1929] Created device

/job:localhost/replica:0/task:0/device:GPU:0 with 10272 MB memory: -> device: 0, name: NVIDIA GeForce RTX 3060, pci bus id: 0000:0a:00.0, compute capability: 8.6

2025-11-27 16:47:45.827539: I

external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful

NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at

<https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

2025-11-27 16:47:45.828680: I

tensorflow/core/common_runtime/gpu/gpu_device.cc:1929] Created device

/job:localhost/replica:0/task:0/device:GPU:1 with 10273 MB memory: -> device: 1, name: NVIDIA GeForce RTX 3060, pci bus id: 0000:0b:00.0, compute capability: 8.6

```
[10]: base_model.trainable = False
```

```

[11]: inputs = tf.keras.Input(shape=(width, height, 3))

[12]: from tensorflow.keras.applications.inception_v3 import preprocess_input

[13]: x = preprocess_input(inputs)

x = base_model(x, training=False)

x = layers.GlobalAveragePooling2D()(x)

x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.5)(x)

outputs = layers.Dense(NUM_CAT_BREEDS, activation="softmax")(x)

model = models.Model(inputs, outputs)

[14]: model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=['accuracy']
)

[15]: IMG_SIZE = (width, height)
BATCH_SIZE = 32

train_ds = tf.keras.utils.image_dataset_from_directory(
    "CatData/_tmp_standardized/train",
    validation_split=.2,
    subset="training",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    "CatData/_tmp_standardized/val",
    validation_split=.2,
    subset="training",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

test_ds = tf.keras.utils.image_dataset_from_directory(
    "CatData/_tmp_standardized/test",
    shuffle=False,

```



```

        image_size=IMG_SIZE,
        batch_size=BATCH_SIZE
    )

    class_names = train_ds.class_names
    NUM_CLASSES = len(class_names)
    print(class_names)

```

Found 2060 files belonging to 5 classes.
 Using 1648 files for training.
 Found 282 files belonging to 5 classes.
 Using 226 files for training.
 Found 258 files belonging to 5 classes.
 ['bengal', 'domestic_shorthair', 'maine_coon', 'ragdoll', 'siamese']

```

[16]: data_augmentation = tf.keras.Sequential([
        tf.keras.layers.RandomFlip("horizontal"),
        tf.keras.layers.RandomRotation(.1),
        tf.keras.layers.RandomZoom(.1)
    ])

```

```

[17]: # Include augmentation in the model
x = data_augmentation(inputs)
x = base_model(x, training=False)

```

```

[18]: # Train the head
import numpy as np
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=20,
    restore_best_weights=True
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=50,
    callbacks=[early_stop]
)

print("Number of epochs run:", len(history.history["loss"]))
print("Best epoch according to callback:", early_stop.best_epoch)
print("Argmin of val_loss:", np.argmin(history.history["val_loss"]))

```

Epoch 1/50

```

2025-11-27 16:47:50.514587: I
external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:454] Loaded cuDNN
version 8800
2025-11-27 16:47:54.699878: I external/local_xla/xla/service/service.cc:168] XLA
service 0x7f8c3cdfd060 initialized for platform CUDA (this does not guarantee
that XLA will be used). Devices:
2025-11-27 16:47:54.699912: I external/local_xla/xla/service/service.cc:176]
StreamExecutor device (0): NVIDIA GeForce RTX 3060, Compute Capability 8.6
2025-11-27 16:47:54.699919: I external/local_xla/xla/service/service.cc:176]
StreamExecutor device (1): NVIDIA GeForce RTX 3060, Compute Capability 8.6
2025-11-27 16:47:54.705514: I
tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269] disabling MLIR
crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRECTORY` to enable.
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1764258474.771867 2670408 device_compiler.h:186] Compiled cluster
using XLA! This line is logged at most once for the lifetime of the process.

52/52 [=====] - 16s 191ms/step - loss: 0.8621 -
accuracy: 0.6693 - val_loss: 0.4584 - val_accuracy: 0.8717
Epoch 2/50
52/52 [=====] - 5s 93ms/step - loss: 0.5175 - accuracy:
0.8167 - val_loss: 0.4402 - val_accuracy: 0.8496
Epoch 3/50
52/52 [=====] - 5s 93ms/step - loss: 0.4455 - accuracy:
0.8228 - val_loss: 0.4012 - val_accuracy: 0.8496
Epoch 4/50
52/52 [=====] - 5s 93ms/step - loss: 0.3876 - accuracy:
0.8671 - val_loss: 0.3620 - val_accuracy: 0.8628
Epoch 5/50
52/52 [=====] - 5s 92ms/step - loss: 0.3387 - accuracy:
0.8908 - val_loss: 0.3457 - val_accuracy: 0.8717
Epoch 6/50
52/52 [=====] - 5s 93ms/step - loss: 0.3168 - accuracy:
0.8968 - val_loss: 0.3419 - val_accuracy: 0.8717
Epoch 7/50
52/52 [=====] - 5s 93ms/step - loss: 0.3049 - accuracy:
0.8932 - val_loss: 0.3334 - val_accuracy: 0.8761
Epoch 8/50
52/52 [=====] - 5s 92ms/step - loss: 0.2726 - accuracy:
0.9084 - val_loss: 0.3478 - val_accuracy: 0.8850
Epoch 9/50
52/52 [=====] - 5s 92ms/step - loss: 0.2518 - accuracy:
0.9138 - val_loss: 0.3335 - val_accuracy: 0.8717
Epoch 10/50
52/52 [=====] - 5s 92ms/step - loss: 0.2507 - accuracy:
0.9181 - val_loss: 0.3475 - val_accuracy: 0.8673
Epoch 11/50

```

52/52 [=====] - 5s 92ms/step - loss: 0.2406 - accuracy:
0.9187 - val_loss: 0.3644 - val_accuracy: 0.8805
Epoch 12/50
52/52 [=====] - 5s 92ms/step - loss: 0.2104 - accuracy:
0.9326 - val_loss: 0.3443 - val_accuracy: 0.8805
Epoch 13/50
52/52 [=====] - 5s 92ms/step - loss: 0.2188 - accuracy:
0.9199 - val_loss: 0.3581 - val_accuracy: 0.8850
Epoch 14/50
52/52 [=====] - 5s 92ms/step - loss: 0.1941 - accuracy:
0.9302 - val_loss: 0.3896 - val_accuracy: 0.8717
Epoch 15/50
52/52 [=====] - 5s 93ms/step - loss: 0.1887 - accuracy:
0.9302 - val_loss: 0.3513 - val_accuracy: 0.8850
Epoch 16/50
52/52 [=====] - 5s 92ms/step - loss: 0.1660 - accuracy:
0.9405 - val_loss: 0.3678 - val_accuracy: 0.8761
Epoch 17/50
52/52 [=====] - 5s 92ms/step - loss: 0.1749 - accuracy:
0.9436 - val_loss: 0.3395 - val_accuracy: 0.8805
Epoch 18/50
52/52 [=====] - 5s 93ms/step - loss: 0.1609 - accuracy:
0.9454 - val_loss: 0.3514 - val_accuracy: 0.8717
Epoch 19/50
52/52 [=====] - 5s 92ms/step - loss: 0.1488 - accuracy:
0.9490 - val_loss: 0.3835 - val_accuracy: 0.8673
Epoch 20/50
52/52 [=====] - 5s 92ms/step - loss: 0.1765 - accuracy:
0.9333 - val_loss: 0.3800 - val_accuracy: 0.8717
Epoch 21/50
52/52 [=====] - 5s 93ms/step - loss: 0.1648 - accuracy:
0.9460 - val_loss: 0.3892 - val_accuracy: 0.8761
Epoch 22/50
52/52 [=====] - 5s 92ms/step - loss: 0.1347 - accuracy:
0.9563 - val_loss: 0.3583 - val_accuracy: 0.8850
Epoch 23/50
52/52 [=====] - 5s 92ms/step - loss: 0.1288 - accuracy:
0.9551 - val_loss: 0.3716 - val_accuracy: 0.8805
Epoch 24/50
52/52 [=====] - 5s 92ms/step - loss: 0.1245 - accuracy:
0.9600 - val_loss: 0.4333 - val_accuracy: 0.8717
Epoch 25/50
52/52 [=====] - 5s 92ms/step - loss: 0.1089 - accuracy:
0.9618 - val_loss: 0.3771 - val_accuracy: 0.8850
Epoch 26/50
52/52 [=====] - 5s 92ms/step - loss: 0.1016 - accuracy:
0.9678 - val_loss: 0.3611 - val_accuracy: 0.8850
Epoch 27/50

```
52/52 [=====] - 5s 94ms/step - loss: 0.0809 - accuracy:
0.9733 - val_loss: 0.3918 - val_accuracy: 0.8938
Number of epochs run: 27
Best epoch according to callback: 6
Argmin of val_loss: 6
```

```
[19]: model.save("cats_feature_extractor.keras")
```

```
[20]: # Fine tuning the top of InceptionV3
base_model.trainable = True

fine_tune_from = 240

for layer in base_model.layers[:fine_tune_from]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

fine_tune_history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=5
)
```

```
Epoch 1/5
52/52 [=====] - 25s 238ms/step - loss: 0.2339 -
accuracy: 0.9181 - val_loss: 0.3161 - val_accuracy: 0.8850
Epoch 2/5
52/52 [=====] - 6s 107ms/step - loss: 0.1613 -
accuracy: 0.9508 - val_loss: 0.2950 - val_accuracy: 0.8982
Epoch 3/5
52/52 [=====] - 6s 107ms/step - loss: 0.0979 -
accuracy: 0.9733 - val_loss: 0.3046 - val_accuracy: 0.8982
Epoch 4/5
52/52 [=====] - 6s 107ms/step - loss: 0.0757 -
accuracy: 0.9800 - val_loss: 0.3013 - val_accuracy: 0.9159
Epoch 5/5
52/52 [=====] - 6s 107ms/step - loss: 0.0492 -
accuracy: 0.9885 - val_loss: 0.3231 - val_accuracy: 0.8982
```

12 MANUAL TEST

```
[21]: def TestImage(imgpath):  
    img = tf.keras.utils.load_img(imgpath, target_size=IMG_SIZE)  
  
    img_array = tf.keras.utils.img_to_array(img)  
    img_array = tf.expand_dims(img_array, 0)  
  
    predictions = model.predict(img_array)[0]  
  
    for name, p in sorted(zip(class_names, predictions), key=lambda x: -x[1]):  
        print(f"{name:20s} {p:.3f}")
```

```
[22]: import glob  
import random  
  
def getRandTestImage(breed):  
    return random.choice(glob.glob(f"CatData/_tmp_standardized/test/{breed}/*"))
```

```
[23]: TestImage(getRandTestImage("ragdoll"))
```

```
1/1 [=====] - 2s 2s/step  
ragdoll          0.998  
maine_coon       0.002  
siamese          0.000  
domestic_shorthair 0.000  
bengal           0.000
```

```
[24]: TestImage(getRandTestImage("maine_coon"))
```

```
1/1 [=====] - 0s 40ms/step  
maine_coon       1.000  
ragdoll          0.000  
domestic_shorthair 0.000  
bengal           0.000  
siamese          0.000
```

```
[25]: TestImage(getRandTestImage("bengal"))
```

```
1/1 [=====] - 0s 26ms/step  
bengal           0.849  
domestic_shorthair 0.149  
maine_coon       0.002  
siamese          0.000  
ragdoll          0.000
```

```
[26]: TestImage(getRandTestImage("siamese"))
```

```
1/1 [=====] - 0s 26ms/step
siamese          1.000
domestic_shorthair 0.000
ragdoll          0.000
bengal           0.000
maine_coon       0.000
```

```
[27]: TestImage(getRandTestImage("domestic_shorthair"))
```

```
1/1 [=====] - 0s 26ms/step
domestic_shorthair 0.998
bengal            0.001
siamese           0.000
maine_coon        0.000
ragdoll           0.000
```

```
[28]: import matplotlib.pyplot as plt

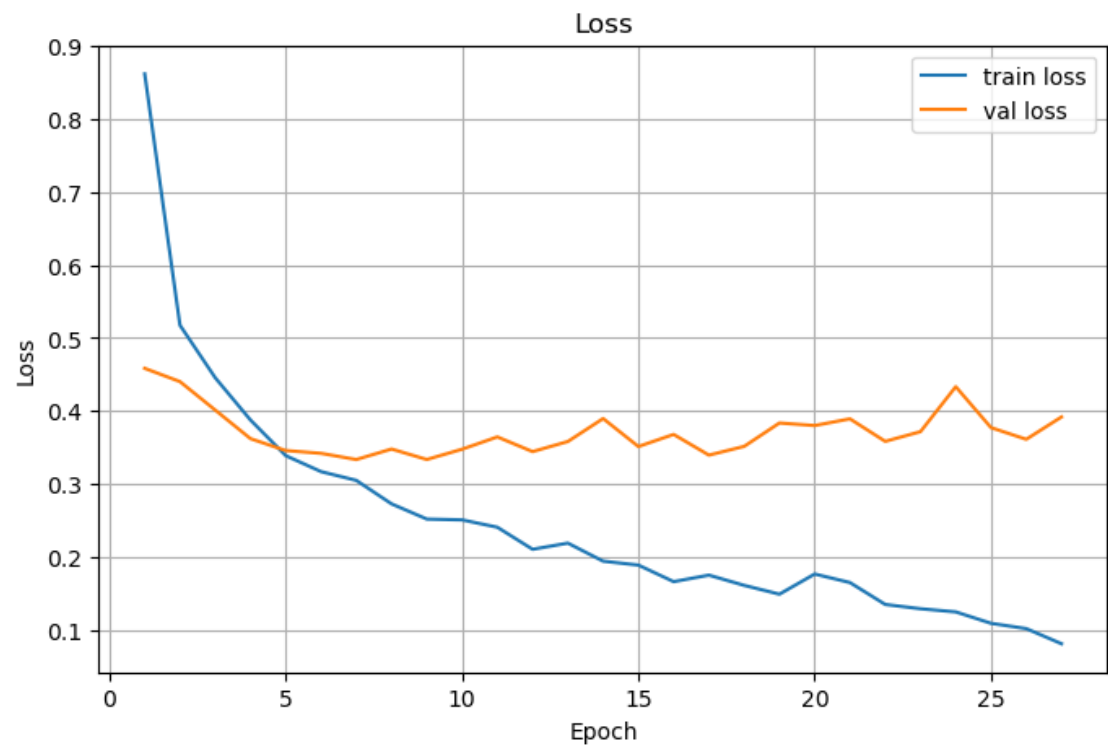
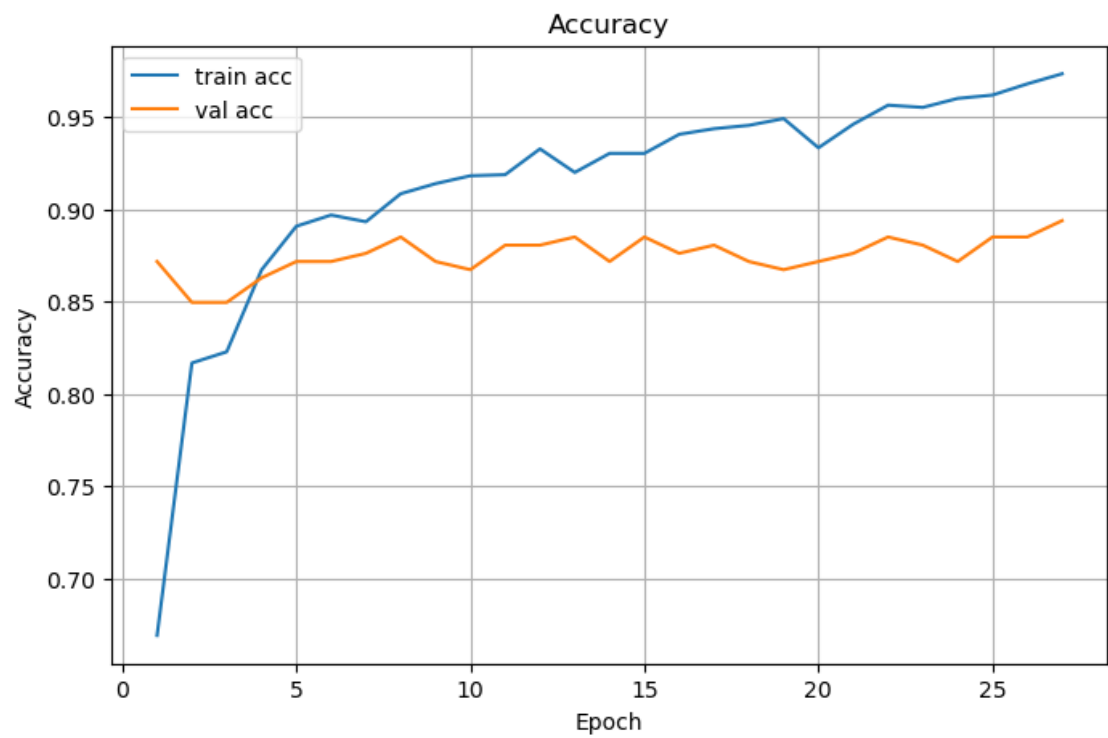
def plot_history(history):
    acc = history.history["accuracy"]
    val_acc = history.history["val_accuracy"]
    loss = history.history["loss"]
    val_loss = history.history["val_loss"]

    epochs = range(1, len(acc) + 1)

    plt.figure()
    plt.plot(epochs, acc, label="train acc")
    plt.plot(epochs, val_acc, label="val acc")
    plt.legend()
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.title("Accuracy")

    plt.figure()
    plt.plot(epochs, loss, label="train loss")
    plt.plot(epochs, val_loss, label="val loss")
    plt.legend()
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("Loss")

plot_history(history)
```



```
[29]: len(history.history["loss"])
```

```
[29]: 27
```

```
[30]: print("Best epoch:", early_stop.best_epoch)
```

Best epoch: 6

```
[31]: import sklearn

# 1. Hent Sande Etiketter (True Labels)
# Samler alle 'y' værdier (labels) fra valideringssættet.
y_true = []
for images, labels in test_ds:
    # labels.numpy() konverterer tf.Tensor til NumPy array
    y_true.extend(labels.numpy())

y_true = np.array(y_true)
print(f"y_true shape: {y_true.shape}")

# 2. Hent Modellens Rå Forudsigelser (Probabilities)
# model.predict tager hele val_ds datasættet som input
y_pred_raw = model.predict(test_ds)
print(f"y_pred_raw shape: {y_pred_raw.shape}")
# Bør være (antal billeder, 5 racer)

# 3. Konverter Sandsynligheder til Klasser
# np.argmax vælger indexet (klassenummeret) med den højeste sandsynlighed
y_pred = np.argmax(y_pred_raw, axis=1)
print(f"y_pred shape: {y_pred.shape}")

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Kør de ovenstående trin 1 og 2 for at få y_true og y_pred...

# 4. Generer Matricen
cm = confusion_matrix(y_true, y_pred)

# 5. Visualiser Matricen
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names # class_names fik du fra val_ds
)
fig, ax = plt.subplots(figsize=(10, 10))
disp.plot(ax=ax, cmap=plt.cm.Blues)
plt.title("Confusion Matrix for Cat Breeds (Validation Set)")
```



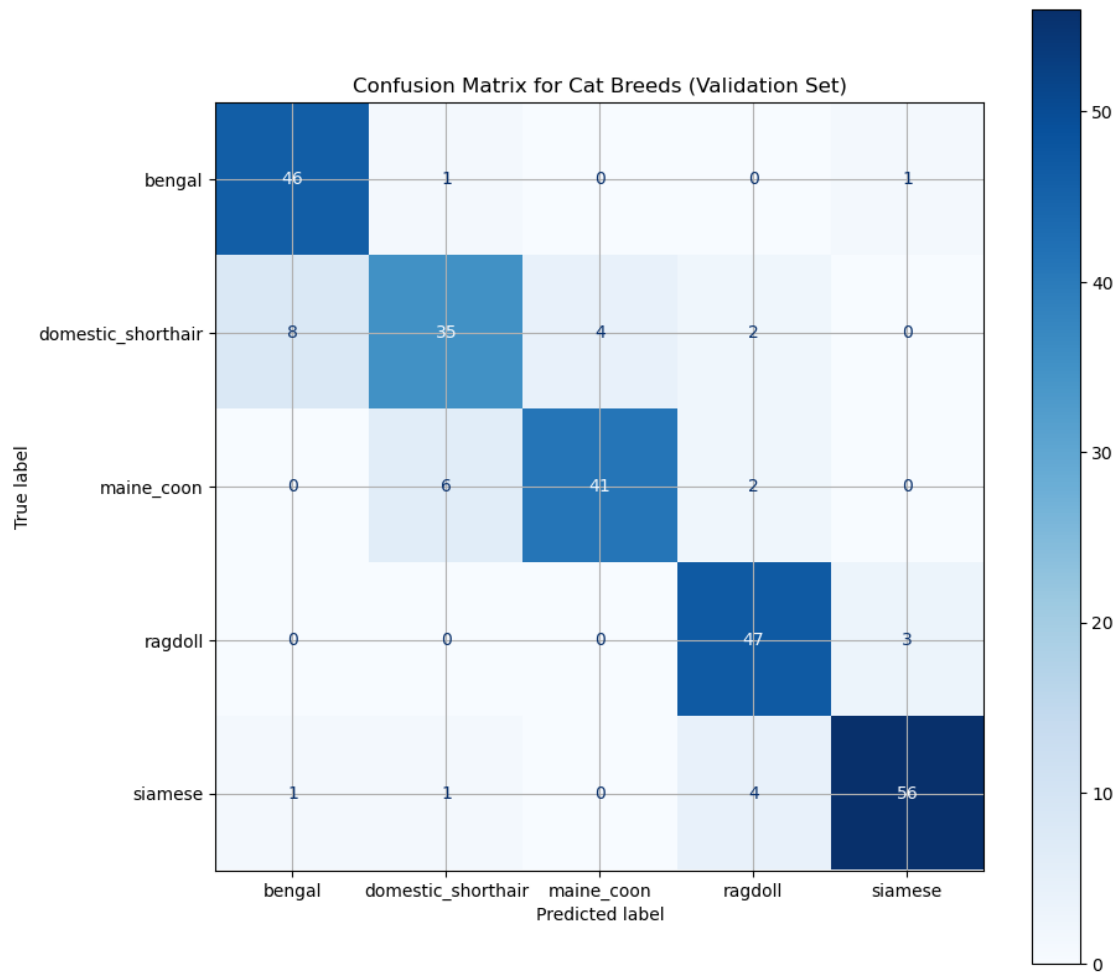
```
plt.show()
```

```
y_true shape: (258,)
```

```
9/9 [=====] - 1s 73ms/step
```

```
y_pred_raw shape: (258, 5)
```

```
y_pred shape: (258,)
```



```
[32]: model.save("cats with fine tuning.keras")
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```